实验三 存储管理实验

一. 目的要求：

1、通过编写和调试存储管理的模拟程序以加深对存储管理方案的理解。熟悉虚存管理的各种页面淘汰算法。

2、通过编写和调试地址转换过程的模拟程序以加强对地址转换过程的了解。

二 . 例题

设计一个请求页式存储管理方案。并编写模拟程序实现之。产生一个需要访问的指令地址流。它是一系列需要访问的指令的地址。为不失一般性，你可以适当地（用人工指定地方法或用随机数产生器）生成这个序列，使得 50％的指令是顺序执行的。25％的指令均匀地散布在前地址部分，25％的地址是均匀地散布在后地址部分。

为简单起见。页面淘汰算法采用 FIFO 页面淘汰算法，并且在淘汰一页时，只将该页在页表中抹去。而不再判断它是否被改写过，也不将它写回到辅存。

具体的做法可以是：

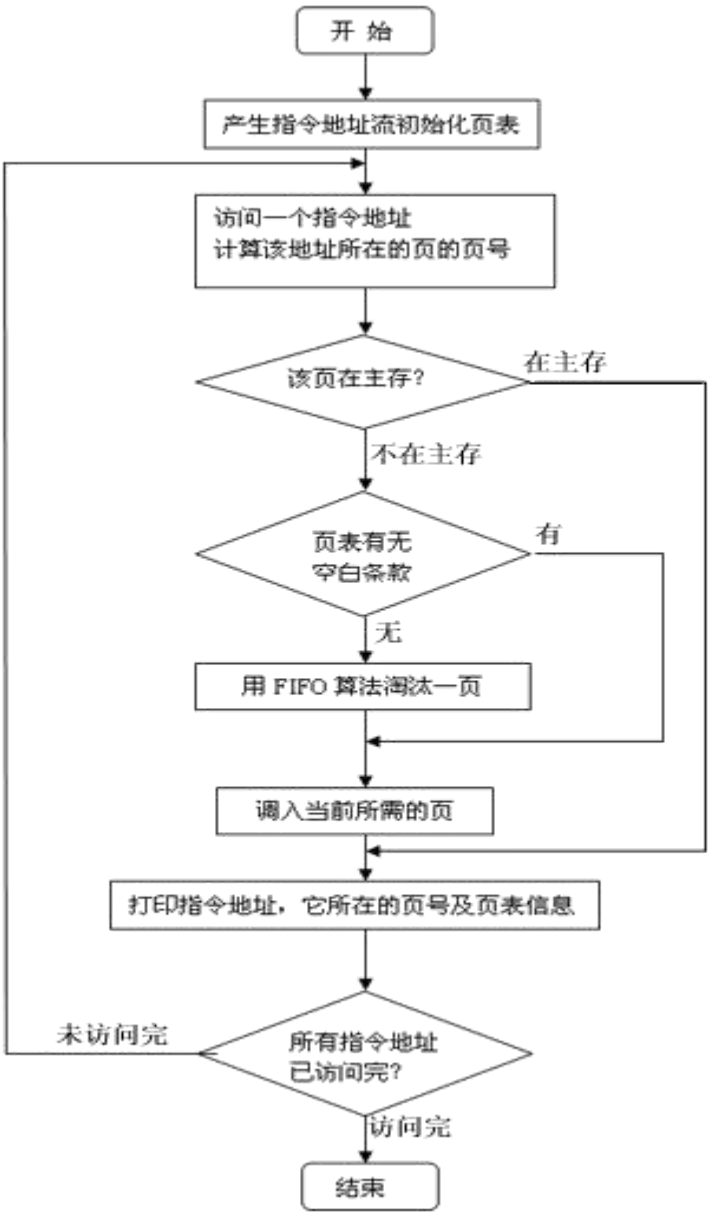
产生一个需要访问的指令地址流； （页号，页内偏移）

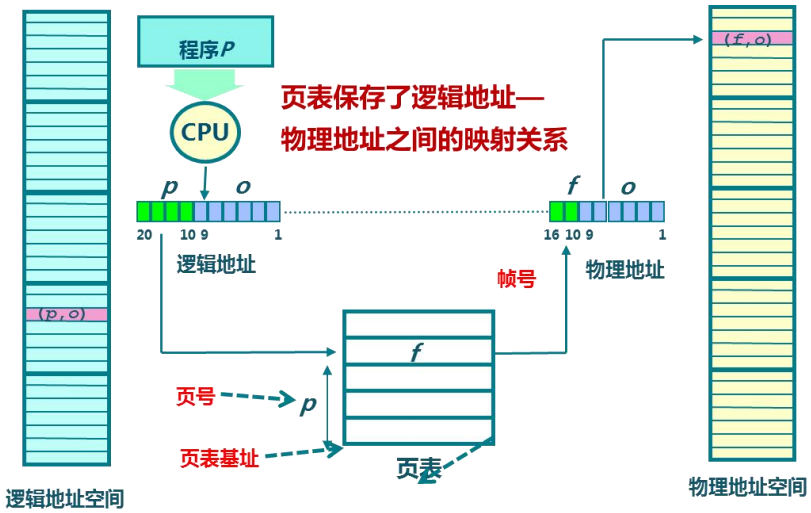
指令合适的页面尺寸（例如以 1K 或 2K 为 1 页）；

指定内存页表的最大长度，并对页表进行初始化；

每访问一个地址时，首先要计算该地址所在的页的页号，然后查页表，判断该页是否在主存——如果该页已在主存，则打印页表情况；如果该页不在主存且页表未满，则调入一页并打印页表情况；如果该页不足主存且页表已满，则按 FIFO 页面淘汰算法淘汰一页后调入所需的页，打印页表情况；逐个地址访问，直到所有地址访问完毕。

存储管理算法的流程图如下：





三． 实验题：

1、设计一个固定式分区分配（连续分配）的存储管理方案，并模拟实现分区的分配和回收过程。

可以假定每个作业都是批处理作业，并且不允许动态申请内存。为实现分区的分配和回收，可以设定一个分区说明表，按照表中的有关信息进行分配，并根据分区的分配和回收情况修改该表。

分区说明表

分区号	大小(KB)	始址(KB)	状态
1	25	16	已分配
2	38	40	已分配
3	57	110	未分配
…	…	…	…

```
#include <iostream>

#include <vector>

#include <queue>

using namespace std;
```

```
typedef struct {
    string name;
    int num;
    int runTime;
    int needTime;
    int size;
    bool done;
} JCB;

typedef struct {
    int workNum;
    int startAdd;
    int quantity;
    bool statue;
} MB;

void input();
bool judge();
void run();
void print();
void recycle(int);

vector<JCB> j;
vector<MB> m;
```

```
queue<JCB> wait;


int countWork,countBlock,time;


// 输入基本信息
void input(){
    cout << "请输入作业数：";
    cin >> countWork;
    JCB tmp;
    for (int i = 0;i < countWork;i++){
        cout << endl;
        cout << "作业No." << i << endl;
        cout << "请输入作业名：";
        cin >> tmp.name;
        cout << "请输入作业执行时间：";
        cin >> tmp.needTime;
        cout << "请输入作业大小：";
        cin >> tmp.size;
        cout << endl;
        tmp.runTime = -1;
        tmp.done = false;
        tmp.num = i;
        j.push_back(tmp);
        wait.push(tmp);
```

```
    }

    cout << "请输入内存分区数：";
    cin >> countBlock;
    MB tmp1;
    for (int i = 0;i < countBlock;i++) {
        cout << endl;
        cout << "分区No." << i << endl;
        cout << "请输入分区大小：";
        cin >> tmp1.quantity;
        cout << "请输入分区开始地址：";
        cin >> tmp1.startAdd;
        tmp1.statue = false;
        m.push_back(tmp1);
    }
}

//  判断是否还有未执行完的作业
bool judge(){
    for (int i = 0;i < j.size();i++){
        if(!j[i].done) return true;
    }
    return false;
}
```

```
// 模拟运行
void run(){
    while(judge()){
        JCB tmp;
        if(!wait.empty()){
            tmp = wait.front();
            wait.pop();
            int i;
            for (i = 0;i < m.size();i++){
                if(m[i].quantity >= tmp.size && m[i].statue == false){
                    m[i].workNum = tmp.num;
                    m[i].statue = true;
                    break;
                }
            }
            if(i >= m.size()) wait.push(tmp);
        }
        for (int i = 0;i < m.size();i++){
            if(m[i].statue){
                j[m[i].workNum].runTime++;
                if (j[m[i].workNum].runTime ==
j[m[i].workNum].needTime){
                    recycle(i);
```

```
                }
            }
        }
        print();
        getchar();
    }
}

// 显示分区信息
void print(){
    cout << "Time:" << ++time << endl;
    cout << "分区号" << "\t 大小" << "\t 始址" <<"\t 状态" << endl;
    for (int i = 0;i < m.size();i++){
        cout << i+1 << "\t" << m[i].quantity << "\t" << m[i].startAdd <<
"\t";
        if(m[i].statue) cout << "已分配" << endl;
        else cout << "未分配" << endl;
    }
}

void recycle(int num){
    j[m[num].workNum].done = true;
    m[num].statue = false;
}
```

```
int main(){
    input();
    run();


    return 0;
}
```

```
分区No.2
请输入分区大小：57
请输入分区开始地址：110
Time:1
分区号  大小    始址    状态
1       25      16      未分配
2       38      40      未分配
3       57      110     已分配
Time:2
分区号  大小    始址    状态
1       25      16      未分配
2       38      40      未分配
3       57      110     已分配

Time:3
分区号  大小    始址    状态
1       25      16      已分配
2       38      40      未分配
3       57      110     已分配

Time:4
分区号  大小    始址    状态
1       25      16      已分配
2       38      40      未分配
3       57      110     未分配

Time:5
分区号  大小    始址    状态
1       25      16      已分配
2       38      40      未分配
3       57      110     已分配

Time:6
分区号  大小    始址    状态
1       25      16      未分配
2       38      40      未分配
3       57      110     已分配

Time:7
分区号  大小    始址    状态
1       25      16      未分配
2       38      40      未分配
3       57      110     已分配

Time:8
分区号  大小    始址    状态
```

2、设计一个**可变式分区分配**的存储管理方案。并模拟实现分区的**分配和回收**过程。

对分区的管理法可以是下面三种算法之一：

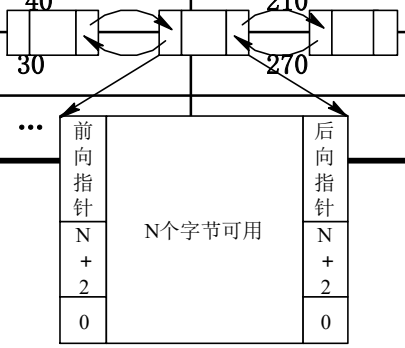
首次适应算法

循环首次适应算法

最佳适应算法

空闲分区表

序号	分区大小（KB ）	分区始址（KB）	状态
1	64	44	可用
2	24	132	可用
3	40	210	可用
4	30	270	可用
5	…		…



P139，P138

首次适应算法：

```
#include <iostream>

#include <queue>

#include <vector>

using namespace std;


typedef struct {
```

```
    string name;
    int numJ;
    int numB;
    int runTime;
    int needTime;
    int size;
    bool done;
} JCB;

typedef struct {
    int workNum;
    int startAdd;
    int quantity;
    int usableQuantity;
    vector<JCB> work;
} MB;

void input();
bool judge();
void run();
void print();
void recycle(int);

vector<JCB> j;
```

```
vector<MB> m;
queue<JCB> wait, r;

int countWork, countBlock, time;

// 输入基本信息
void input() {
    cout << "请输入作业数：";
    cin >> countWork;
    JCB tmp;
    for (int i = 0; i < countWork; i++) {
        cout << endl;
        cout << "作业No." << i << endl;
        cout << "请输入作业名：";
        cin >> tmp.name;
        cout << "请输入作业执行时间：";
        cin >> tmp.needTime;
        cout << "请输入作业大小：";
        cin >> tmp.size;
        cout << endl;
        tmp.runTime = -1;
        tmp.done = false;
        tmp.numJ = i;
        j.push_back(tmp);
```

```
        wait.push(tmp);
    }

    cout << "请输入内存分区数：";
    cin >> countBlock;
    MB tmp1;
    for (int i = 0; i < countBlock; i++) {
        cout << endl;
        cout << "分区 No." << i << endl;
        cout << "请输入分区大小：";
        cin >> tmp1.quantity;
        cout << "请输入分区开始地址：";
        cin >> tmp1.startAdd;
        tmp1.usableQuantity = tmp1.quantity;
        m.push_back(tmp1);
    }
}

//  判断是否还有未执行完的作业
bool judge() {
    for (int i = 0; i < j.size(); i++) {
        if (!j[i].done)
            return true;
    }
```

```
    return false;
}

// 模拟运行
void run() {
    while (judge()) {
        JCB tmp;
        if (!wait.empty()) {
            tmp = wait.front();
            wait.pop();
            bool flag = false;
            for (int i = 0; i < m.size(); i++) {
                if (tmp.size <= m[i].usableQuantity) {
                    m[i].usableQuantity -= tmp.size;
                    m[i].work.push_back(tmp);
                    j[tmp.numJ].numB = i;
                    r.push(tmp);
                    flag = true;
                    break;
                }
            }
            if(!flag) wait.push(tmp);
        }
        for (int i = 0; i < r.size(); i++) {
```

```
                JCB tmp = r.front();
                r.pop();
                tmp.runTime++;
                if (tmp.runTime == tmp.needTime) {
                    recycle(tmp.numJ);
                } else {
                    r.push(tmp);
                }
            }
            print();
            getchar();
        }
    }

    // 显示分区信息
    void print() {
        cout << "Time:" << ++time << endl;
        cout << "分区号"
             << "\t 可用大小"
             << "\t 始址"
             << "\t 容纳作业" << endl;
        for (int i = 0; i < m.size(); i++) {
            cout << i << "\t" << m[i].usableQuantity << "\t\t" <<
m[i].startAdd
```

```
            << "\t";
        for (vector<JCB>::iterator it = m[i].work.begin();
             it != m[i].work.end(); it++) {
            cout << (*it).name << " ";
        }
        cout << endl;
    }
}

void recycle(int num) {
    j[num].done = true;
    m[j[num].numB].usableQuantity += j[num].size;
    for (vector<JCB>::iterator it = m[j[num].numB].work.begin();
         it != m[j[num].numB].work.end(); it++) {
        if ((*it).numJ == num) {
            m[j[num].numB].work.erase(it);
            break;
        }
    }
}

int main() {
    input();
    run();
```

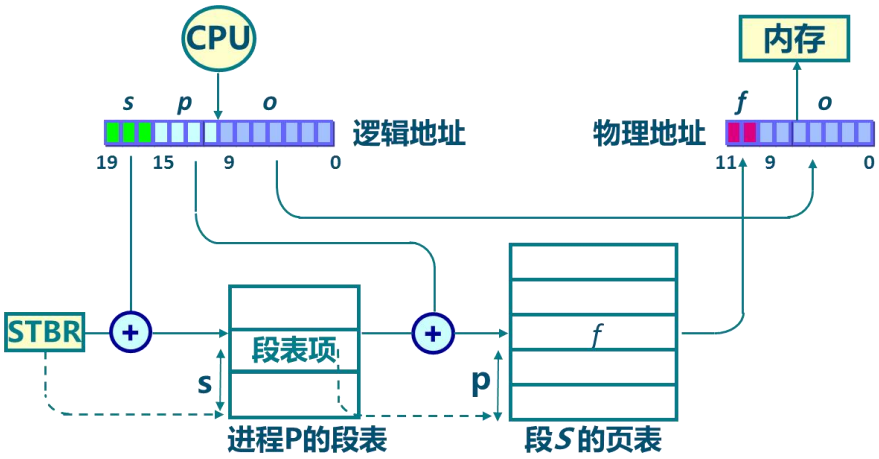
```
    return 0;
}
```

```
Time:1
分区号  可用大小        始址    容纳作业
0       8               1       a
1       17              20
Time:2
分区号  可用大小        始址    容纳作业
0       8               1       a
1       7               20      b

Time:3
分区号  可用大小        始址    容纳作业
0       1               1       a c
1       7               20      b

Time:4
分区号  可用大小        始址    容纳作业
0       13              1       c
1       7               20      b

Time:5
分区号  可用大小        始址    容纳作业
0       13              1       c
1       17              20

Time:6
分区号  可用大小        始址    容纳作业
0       13              1       c
1       17              20

Time:7
分区号  可用大小        始址    容纳作业
0       20              1
1       17              20
```

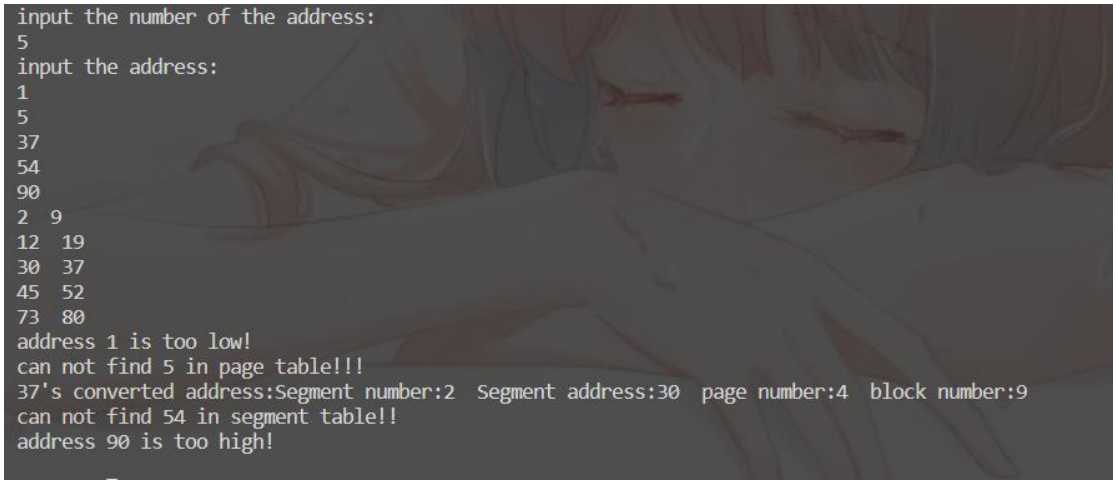
3、编写并调试一个**段页式存储管理**的地址转换的模拟程序。

首先设计好段表、页表，然后给出若干个有一定代表性的地址，通过查找段表页表后得到转换的地址。

要求打印转换前的地址，相应的段表，页表条款及转换后的地址，以便检查。




```
#include <cstring>
#include <iostream>
#define Maxn 5
using namespace std;

int page_table[5] = {2, 3, 6, 8, 9};
int Seg_address[5] = {2, 12, 30, 45, 73};
bool page_state[10];
int address[100];
int tran_address[100];
int num;
int Seg_table_register[2];

typedef struct {
	int seg_address;
	int seg_final_address;
} Box;

typedef struct {
	Box data[Maxn];
} Segment;

Segment seg_table;

void Seg_table() {
	Seg_table_register[0] = Seg_address[0];
```

```
    Seg_table_register[1] = Maxn;
    memset(page_state, false, sizeof(page_state));
    for (int i = 0; i < 5; i++) {
        seg_table.data[i].seg_address = Seg_address[i];
        seg_table.data[i].seg_final_address =
            page_table[4] + Seg_address[i] - Seg_table_register[0];
        page_state[page_table[i]] = true;
        cout << seg_table.data[i].seg_address << "    "
             << seg_table.data[i].seg_final_address << endl;
    }
}

void init() {
    cout << "input the number of the address:" << endl;
    cin >> num;
    cout << "input the address:" << endl;
    for (int i = 0; i < num; i++)
        cin >> address[i];
    Seg_table();
}

void address_translation() {
    init();
    for (int k = 0; k < num; k++) {
        for (int i = 0; i < Maxn; i++) {
            if ((address[k] >= seg_table.data[i].seg_address) &&
                (address[k] <= seg_table.data[i].seg_final_address)) {
                int index = 0;
                for (int j = 0; j < 5; j++) {
                    if (seg_table.data[i].seg_address + page_table[j] -
                                Seg_table_register[0] ==
                            address[k] &&
                        page_state[page_table[j]] == true) {
                        cout << address[k] << "'s converted address:";
                        cout << "Segment number:" << i
                             << "    Segment address:" << Seg_address[i]
                             << "    page number:" << j
                             << "    block number:" << page_table[j] << endl;
                        page_state[page_table[j]] = false;
                        index = 1;
                        i = Maxn;
                    }
                }
                if (index == 0) {
```

```
                    cout << "can not find " << address[k] << " in page table!!!"
                            << endl;
                    i = Maxn;
                }
            } else if (address[k] < Seg_table_register[0]) {
                cout << "address " << address[k] << " is too low!" << endl;
                i = Maxn;
            } else if (address[k] >
                        seg_table.data[Maxn - 1].seg_final_address) {
                cout << "address " << address[k] << " is too high!" << endl;
                i = Maxn;
            } else {
                if (i == Maxn - 1)
                    cout << "can not find " << address[k]
                            << " in segment table!!" << endl;
            }
        }
    }
}

int main() {
    address_translation();
    return 0;
}
```